

Feature Selection using Particle Swarm Optimization and Ensemble-based Machine Learning Models for Ransomware Detection

Neel Kumar Yadav Gurukala (✉ neelkumar.003@gmail.com)

Liverpool John Moores University

Deepak Kumar Verma

Upgrad Education Pvt. Ltd

Research Article

Keywords: Ransomware, Machine learning, Ensemble classifiers, Feature Selection, Particle Swarm Optimization

Posted Date: November 16th, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-3604834/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Feature Selection using Particle Swarm Optimization and Ensemble-based Machine Learning Models for Ransomware Detection

Neel Kumar Yadav Gurukala · Deepak Kumar Verma

Received: date / Accepted: date

Abstract Ransomware attacks have become a major threat to organizations and individuals, as such an attack can cause significant financial loss and disruption to business operations. Traditional methods of ransomware detection, such as signature-based detection and heuristic-based detection, have proven to be inadequate in dealing with the constantly evolving ransomware variants.

Machine learning (ML)-based detection methods have shown promise in detecting ransomware. These methods rely on the extraction of relevant features from the samples and the training of a classifier to distinguish between ransomware and non-ransomware samples. Due to the high dimensionality of the feature space, machine learning algorithms can be employed to identify a crucial subset of features which in turn enhances the detection accuracy.

This research presents a novel approach that combines ensemble classifiers with feature selection using the Particle Swarm Optimization (PSO) algorithm. The objective is to improve the detection accuracy and reduce false positives and false negatives in classification tasks. Two separate ensemble models were constructed: one comprising Random Forest (RF) and Support Vector Machine (SVM) classifiers, and the other consisting of Decision Tree (DT) and K-Nearest Neighbors (KNN) classifiers. The PSO algorithm was employed to determine the optimal features and their corresponding weights for each ensemble classifier.

Neel Kumar Yadav Gurukala
Liverpool John Moores University, Liverpool, United Kingdom
ORCID: 0009-0001-9896-4005
E-mail: neelkumar.003@gmail.com

Deepak Kumar Verma
Upgrad Education Pvt. Ltd, Mumbai, India
ORCID: 0000-0002-4745-1752
E-mail: deepak.verma1980@gmail.com

Experiments were conducted to evaluate the performance of the proposed approach and the results demonstrated that integrating PSO for feature selection significantly enhanced the overall detection rate compared to using all features with equal weights. By identifying the most relevant features and assigning appropriate weights, the ensemble classifiers achieved higher accuracy and improved the overall classification performance.

Keywords Ransomware · Machine learning · Ensemble classifiers · Feature Selection · Particle Swarm Optimization

1 Introduction

Ransomware is malicious software that locks a person's data and requests payment for unlocking it. It has become a significant threat to both businesses and individuals due to its potential to cause significant monetary harm and impede day-to-day business activities.

1.1 Background of the Study

The origin of ransomware can be traced to the late 1980s and early 1990s, when the earliest cases of ransomware, referred to as "AIDS" or "PC Cyborg," were uncovered. These early forms of ransomware were unsophisticated and were distributed via floppy disks.

In the 2000s, the emergence of the internet and the increasing use of personal computers and smartphones led to a significant increase in the number of ransomware attacks. These attacks primarily targeted individuals and small businesses, and the ransom demands were typically in the range of a few hundred dollars. The most prevalent form of ransomware during this time was known as "scareware,"

which displayed fake alerts and warning messages to scare the victim into paying the ransom.

In the 2010s, ransomware evolved to become more sophisticated and targeted organizations, particularly in the healthcare and financial sectors. These attacks often resulted in the loss of sensitive data and significant financial losses. The most notable ransomware attack of this decade was the WannaCry attack in 2017 [1], which affected over 200,000 computers in 150 countries and caused widespread disruption to business operations.

The COVID-19 pandemic has led to a surge in ransomware attacks, as more people have come to rely on computers and online businesses while working from home. One notable example of this was the attack on Colonial Pipelines in May 2021, which caused disruptions to the major fuel supply chain in seventeen states, including Washington D.C. The company was forced to pay a ransom of around 4.4 million US Dollars [2, 3].

Today, the threat of ransomware continues to be a major concern for organizations and individuals, as attackers increasingly employ advanced tactics such as encryption and targeted attacks. Furthermore, the anonymity provided by the use of cryptocurrency for ransom payments poses additional challenges for detection and prevention, as well as for law enforcement efforts. The use of locally generated encryption keys without communicating to Command-and-Control (C&C) servers also makes these attacks much harder to detect.

Despite the availability of various methods for detecting malware, ransomware - a subset of malware - is still included in the same detection techniques. However, traditional malware detection methods are insufficient in detecting ransomware because it is challenging to distinguish between a benign software application and a ransomware, and they could lock access to crucial data permanently unless the ransom is paid. Additionally, ensuring complete zero-day protection against ransomware attacks, which are constantly evolving and made easier with the advent of Ransomware as a Service (RaaS) [4], is not possible with current detection methods.

Traditional methods of ransomware detection, such as signature-based detection and heuristic-based detection, have proven to be inadequate in dealing with the constantly evolving ransomware variants due to their inherent limitations [5, 6]:

- They can only detect known ransomware, so they are not effective against new or unknown forms of ransomware.
- They can be bypassed by attackers who use techniques such as code obfuscation, which makes it difficult to match the malware to a known signature.
- They are susceptible to "zero-day" attacks, which take advantage of previously undisclosed weaknesses.

- They can generate a high number of false positives and negatives, which can lead to a high number of false alarms and missed threats.

Machine learning-based detection methods, on the other hand, have shown promise in detecting new variants of ransomware [7]. However, machine learning-based detection techniques are often hindered by the high dimensionality of the feature space and the limited size of the available training dataset.

1.2 Problem Statement

Ransomware attacks are becoming increasingly sophisticated and challenging to detect. Traditional signature-based approaches and rule-based methods are often ineffective in detecting ransomware attacks that use sophisticated evasion techniques.

Using machine learning models for detecting ransomware has its own set of challenges mostly with identifying the critical features required to classify a ransomware, the algorithms used for classification and how they are used. This is apart from collecting a diverse dataset for training the ML models.

Furthermore, selecting the most suitable classification algorithm vastly affects the detection accuracy. Each classification algorithm also has its own set of limitations, such as overfitting or underfitting, and might not be suitable in all the cases. Combining multiple classification algorithms reduces these issues to some extent, where the final classification is derived from voting of individual classifications given the based classifiers.

In our literature review, we found that most researchers used a single classifier to detect ransomware. In the studies that proposed using ensemble-based classifier models, the features were selected manually. In other studies that used ensemble-based classifier models, only static features of ransomware were used. As a result, there was a need to develop a more effective and efficient approach to detecting ransomware attacks.

1.3 Aim and Objectives

Evaluate the performance of PSO algorithm for selection of both static and behavioral features in ransomware and using that derived feature set evaluate the performance of algorithms like RF, SVM, DT and KNN in an ensemble-based ML model.

The research objectives are formulated based on the aim of this study which are as follows:

- To evaluate the performance of a PSO based algorithm for critical feature selection.

- To evaluate the efficacy of an ensemble-based ML models by combining RF and SVM and DT and KNN.

1.4 Research Questions

How can we improve the existing ransomware detections techniques using advanced ML algorithms for critical feature selection and classification?

1.5 Scope of the Study

The goal of this research is to enhance the precision of ransomware attack detection while reducing rate of false positives by investigating the use of PSO-based feature selection method in an ensemble-based machine learning model composed of RF, SVM, DT and KNN. However, because of limitations in resources, the proposed model is trained and evaluated using a limited set of both ransomware and non-malicious samples. Therefore, the findings and conclusions of this study are only applicable to this specific dataset. It's noteworthy that while this approach may be effective for other types of malwares, it has not been studied or validated in this research.

1.6 Significance of the Study

Ransomware attacks pose a significant threat to businesses, as they can result in financial losses, data breaches, and disruptions to operations. As ransomware attacks evolve and become more advanced, it becomes increasingly important to develop new techniques to detect and respond to them. Additionally, ransomware can damage a company's reputation by raising concerns about their ability to protect customer data. The field of ransomware detection is constantly evolving, with ongoing research and development efforts to create new and more effective techniques. This research aims to identify any deficiencies in current detection and feature selection techniques for ransomware, and to suggest improvements. To accomplish this, the study employs advanced machine learning algorithms for feature selection and classification and then compares against the existing research.

1.7 Structure of the Study

This paper starts with an introduction to ransomware and a survey of recent ransomware attacks. It presents problems with ransomware detection and proposes a method of using PSO algorithm with a wrapped ensemble classification algorithms. It then outlines a review of recent research done

on various types of detection techniques and challenges in each of the techniques and how the researchers have tried to overcome them. This is followed by the research methodology being used in this study and the implementation of such a methodology. This is continued by validation and discussion of results of the experiments and finally with a conclusion on the study and any possible directions for future research.

2 Literature Review

This literature review will provide an overview of recent research on ransomware, including the types, the motivation behind the attacks, its characteristics, propagation mechanisms, prevention techniques and lastly will try to find limitations in each of those techniques.

2.1 Types of ransomware

There are several types of ransomware based on the type of attack they execute and how a victim is targeted:

Crypto ransomware : This is a common type of ransomware which encrypts the victim's files and demands a ransom payment to unlock them. It can target any type of file on the victim's computer, including documents, images, and videos.

Locker ransomware : Locker ransomware denies the victim access to their computer by locking the screen or disabling the keyboard. It can target both desktop and mobile devices.

Doxware : Doxware [8] threatens to publish the victim's personal data or sensitive information unless a ransom is paid. It can target any type of sensitive data, including financial information, medical records, and personal photos.

Popcorn Time ransomware : This is unique in that it gives the victim two options to regain access to their encrypted files. The first option is to pay the ransom, usually in the form of Bitcoin. The second option, however, is to help spread the ransomware to other potential victims. The victim is instructed to share a link to the ransomware with two or more people, and if those people become infected and pay the ransom, the victim's files will be decrypted for free.

Mobile ransomware : Mobile ransomware targets smartphones and tablets by encrypting files or locking the device. It can be delivered through malicious apps, phishing messages, or compromised websites.

Screen locker ransomware : Similar to locker ransomware, screen locker ransomware locks the victim's computer screen, making it impossible to access any files or programs.

Master boot record (MBR) ransomware : MBR ransomware infects the master boot record of a victim's computer, preventing it from booting up properly. It typically targets Windows computers.

Hybrid ransomware : Hybrid ransomware combines the characteristics of different types of ransomware. For example, it may encrypt the victim's files and lock their screen at the same time.

Crypto-jacking ransomware : Crypto-jacking ransomware hijacks the victim's computer to mine cryptocurrencies, and then demands a ransom payment to stop the attack.

This list is not comprehensive. The types of ransomware and their methods of operation are constantly evolving due to the various motivations behind ransomware attacks and the increasing popularity of ransomware-as-a-service (RaaS) [4, 9].

2.2 Motivation behind the ransomware attacks

One of the primary motivations behind ransomware attacks is financial gain. Cybercriminals use ransomware to gain access to sensitive data and hold it hostage until a ransom is paid. The ransom demand is usually made in cryptocurrency, making it difficult to trace the money back to the perpetrator.

The recent examples include: WannaCry in 2017 [10, 11, 12] which affected over 250,000 computers in 150 countries, exploiting a vulnerability in Windows systems. During June of 2017, Petya/NotPetya ransomware attack also exploited a vulnerability in Windows systems and spread rapidly through corporate networks, affecting companies such as Maersk, FedEx, and Merck. It caused an estimated \$10 billion in damages. In May of 2019, the city of Baltimore, Maryland, was hit by a ransomware attack that paralyzed its computer systems and forced the city to shut down essential services. The attackers demanded a ransom of \$76,000 in Bitcoin. During May of 2021, the Colonial Pipeline, which supplies fuel to the eastern United States, was hit by a ransomware attack that forced it to shut down operations for several days. The attackers demanded a ransom of \$4.4 million in Bitcoin.

Ransomware attacks can also be motivated by political agendas. State-sponsored hackers may use ransomware attacks to disrupt the operations of foreign governments, organizations, or critical infrastructure, such as hospitals or energy grids. These attacks can be used as a means of political leverage or to cause chaos and destabilize the target country.

Lack of cybersecurity infrastructure and easy access of new ransomware via RaaS also contribute to the increase in ransomware attacks.

2.3 Propagation Mechanisms of Ransomware

Ransomware can propagate through various mechanisms, including email attachments, malicious websites, and exploit kits. Email attachments are the most common propagation mechanism for ransomware, with attackers using social engineering tactics to trick users into downloading and executing the malware. The ransomware, masquerading as authentic MS Office files, may either execute the embedded ransomware code or stealthily download the ransomware in the background through a Trojan delivered by malicious macros embedded within the files [13, 14].

Malicious websites can also distribute ransomware through malvertising/ malicious advertisements [15, 16, 17] that are infected with malware or contain links to malicious websites. These ads are designed to exploit vulnerabilities in software to deliver malware to users' devices, leading to various malicious activities.

Ransomware can also be delivered through drive-by downloads, which refers to the inadvertent downloading of malware while browsing the internet. Often, malware is concealed within legitimate software, so when a user downloads the legitimate software, the malware is also downloaded without their knowledge. Drive-by downloads can happen when opening email attachments, clicking on untrustworthy pop-up ads, or clicking on links that lead to malicious websites [18].

The attackers can also take advantage of vulnerabilities in the operating system or software application that can be exploited to inject malware into the system. This can be due to programming errors, design gaps, or other inherent bugs in the software or system. The 2017 WannaCry ransomware attack is an infamous example of such a propagation technique, where a vulnerability in Microsoft Server Message Block 1.0 (SMB) server [19] was exploited to propagate the ransomware across systems within the network [20].

Removable storage devices, such as USB drives or external hard drives, is also a means of spreading ransomware. When the infected device is connected to another system, the ransomware may execute and propagate to that system. Organizations often lack crucial Cyber Security policies to prevent employees from connecting external devices to the laptops and other systems on the enterprise which then becomes an easy target for attackers [21].

Ransomware can also be distributed through botnets, which are networks of compromised computers that are controlled by cybercriminals. Botnets can be used to deliver ransomware to a large number of victims simultaneously [22, 23].

Watering hole attacks: Ransomware can be distributed through watering hole attacks, where cybercriminals compromise websites that are commonly visited by the target victims. When the victims visit these compromised websites, they may unknowingly download and execute the ransomware [24, 25].

The evolution of social engineering has expanded the range of attack vectors. For example, social engineering can now target individual employees or groups of employees in non-IT roles, using specific information to craft spear phishing attacks. Social engineering can also be employed in other types of attacks, reducing the likelihood of detection by minimizing suspicion.

2.4 Existing Detection Techniques

There are different kinds of techniques aimed at detection and prevention of ransomware attacks. These include the traditional Signature or Heuristic rules-based detection techniques and the machine learning or the recent deep learning classification techniques and lastly network based detection.

2.4.1 Traditional ransomware detection techniques

These are similar to their counterparts in malware detection in their approach to detecting ransomware. These work on comparing the signature and/or heuristic of a ransomware or benign process to a known list of ransomware signatures or behavior. While these techniques are very effective in early detection of known ransomware attacks, the emergence of new types of ransomware that employ obfuscation and adjust their behavior dynamically upon detecting the presence of antivirus software has proven to be a challenge for such techniques [26, 27, 28, 29, 30]. Moreover, existing research has proved that traditional detection techniques often fall short of detecting zero-day attacks [31, 32, 33].

2.4.2 Machine learning-based detection techniques

Machine learning-based detection techniques utilize the various ML algorithms and classification algorithms in overcoming the problems found in the traditional methods. These ML models are first trained on diverse set of known ransomware and are then tested against both old and new forms of ransomware.

With the advances in machine learning algorithms and their growing popularity, various machine learning techniques and algorithms were used in detection of ransomware and in general the different types of malwares. These techniques work in several steps: dataset collection, feature extraction, feature engineering, feature selection, model training and finally model testing and deployment.

Each of these steps has a plethora of machine learning techniques and algorithms to achieve that step's goals. Furthermore, some of these steps can also be done manually or must be done manually. For example, dataset collection is a crucial step which mostly needs manual effort, where you need to find a set of ransomware and benign samples which are then used for training and testing the models. The researchers can use various platforms for collecting these datasets. A lot of research has gone into the other steps and, usually the researchers focus on a particular stage and utilize already established research work for the other steps.

Ucci et al [34], presented a comprehensive survey on the taxonomy of ransomware, the types of ransomware analysis, types of detection techniques and an overall picture of how various machine learning techniques have been used in the detection of ransomware. Alqahtani and Sheldon [7], also presented a survey of existing research related to ransomware early detection techniques and emphasized the key challenges in achieving precise detection of ransomware, such as feature extraction, feature selection and training the ML models. Shetty and Praveen [6], in their research concluded that machine learning-based detection methods outperformed in detection of moderately complex or sophisticated malware.

The common approach for feature extraction is to perform static analysis of ransomware samples and their attack patterns and then identify a list of most important features. Poudyal et al. [35], proposed a reverse engineering framework to perform multi-level analysis of binary files to extract features such as assembly codes, libraries, and function calls. The framework uses a fixed set of features extracted from binary files, which may not be comprehensive or representative of all ransomware variants or families. This also ignores the behavioral aspect of the ransomware and might not perform well against ransomware employing anti-analysis techniques.

Manual feature extraction and selection are time-consuming and prone to human errors. It is challenging to determine which features are relevant to the classification task, and manual selection may overlook important features that are not immediately apparent to human analysts. Additionally, manual selection can become a bottleneck in the machine learning pipeline when dealing with large datasets. Furthermore, manual selection may not capture the subtle relationships between features that are crucial for accurate classification. The selection process may also be biased towards the knowledge and experience of the individual performing the selection, leading to suboptimal performance. The ever-increasing threat of new types of ransomware has also encouraged more researchers to utilize machine learning and deep learning algorithms for feature extraction and selection. Azeez et al. [36], proposed a detection technique by using Principal Component

Analysis (PCA) to perform dimensionality reduction on static features of ransomware samples. They then employed stacked single and ensemble classifiers and later used a generalizer to get the final classification.

Ali et al. [37], proposed a malware detection system, called MALGRA, performs dynamic analysis of binary files to extract Indicators of Compromise (IOCs) for malicious files, which are represented using N-grams. The system also uses TF-IDF to identify the most significant N-grams features for training a machine learning algorithm. Although they confirmed that their proposed model outperformed other studies in similar areas, their dataset was limited and does not include any ransomware samples. Al-Rimy et al. [38], proposed a similar method called Dynamic Pre-encryption Boundary Delineation and Feature Extraction (DPBD-FE), but one, which dynamically determines the boundary of the pre-encryption phase for each attack instance based on the first occurrence of any cryptography-related APIs. Then, it uses an annotated Term Frequency-Inverse Document Frequency (aTF-IDF) technique to extract features from runtime data generated during the pre-encryption phase. They compared their research using different classifier algorithms and confirmed that Random Forest matches and, in some cases, outperforms other classification algorithms.

Al-Rimy et al. [39], also proposed Redundancy Coefficient Gradual Upweighting (RCGU) technique for feature selection and reducing redundant features from the early stages of the ransomware attack to improve the detection efficacy. Choi et al. [40], proposed a method for extracting features from malicious files based on attention mechanism which assigns different weights to different parts of the input data, based on their importance for the output task. The authors have used the attention mechanism to identify the most important application program interface (API) system calls that are used by malware. The authors use a deep learning model that consists of an encoder which converts the sequence of API system calls into a vector representation, and then a decoder which generates an output label (benign or malicious) based on the vector representation. The attention mechanism is applied between the encoder and the decoder, so that the decoder can focus on the most relevant parts of the encoder output.

Gera et al. [41], proposed a hybrid approach to detect and analyze Android ransomware using dominant feature selection algorithm, which uses a combination of correlation coefficient and mutual information to rank the features according to their relevance and redundancy. This was done to extract the most important features from the static and dynamic analysis of Android ransomware which include permissions, intents, API system calls, and network activities.

Rafique et al. [42], proposed a deep learning-based malware detection technique that uses byte and ASM files to engineer features and classify malware families. The approach utilizes two Deep Convolutional Neural Networks to extract features from byte files, and a wrapper-based mechanism selects important opcode features using Support Vector Machine as a classifier. Finally, a Multilayer Perceptron is trained using the hybrid feature space to classify nine different malware families. Similarly, Ashraf et al. [43], proposed a method for detecting ransomware based on feature engineering and deep neural networks using static and dynamic analysis of ransomware to extract features such as registry changes, API calls and DLLs. The authors used different feature selection methods, such as mutual information and principal component analysis, to select the most relevant and non-redundant features. They further used the N-Gram technique to identify the most important sequence of operations which are unique to ransomware, such as Registry Delete operation which follows a specific and repeated sequence in case of a malware, is not observed while monitoring a benign application. The authors use different deep learning models, such as convolutional neural network, recurrent neural network with long short-term memory cells, and transfer learning, to classify the files as benign or ransomware providing more insight into the behavior and characteristics of ransomware families.

Aurangzeb et al. [44], proposed usage of big data techniques for further improving ransomware detection. The authors presented BigRC-EML, a method for ransomware detection and classification based on several static and dynamic features by using ensemble machine learning methods on big data. The authors also optimized feature selection by using Principal Component Analysis (PCA) technique.

Another approach to improving the feature extraction is to visualize the ransomware in greyscale as an image and extract features using machine learning or deep learning algorithms. The research by Xiao et al. [45], proposed a novel framework for malware classification that uses deep Convolutional Neural Networks (CNNs) to automatically extract features from raw binary files without any manual preprocessing or domain knowledge. Their framework named as an effective malware classification framework (MalFCS) visualizes malware as entropy graphs based on structural entropy and extracts features using deep convolutional neural networks. Vasan et al. [46], proposed Image-based Malware Classification using Ensemble of CNNs (IMCEC), which entails applying knowledge from various well-known CNN architectures that have been trained on ImageNet data to malware images and fine-tuning them accordingly. The transferred features were used to train a variety of multiclass classifiers, and to

improve the accuracy of the classification of families of unidentified malware samples posterior probabilities were fused. Yan et al. [47], proposed MalNet, which is a similar method for detecting malware families based on deep neural network and ensemble learning by converting executable files into grayscale images, and then using a Convolutional Neural Network (CNN) to extract features from the images. The authors also extracted opcode sequences from the executable files using a disassembler, and used a Recurrent Neural Network (RNN) with Long Short-Term Memory (LSTM) cells to learn from the opcode sequences. The authors combined the outputs of the CNN and LSTM models and then used them in an ensemble machine learning model comprising of various classification algorithms.

However, visualizing ransomware to extract most important features are limited in their ability to identify malware variants since opcodes can only reflect part of the behavioral semantics. This makes them sensitive to opcode changes in malware variants. Furthermore, the classical CNN requires a fixed input image size. Hence input images need to be cropped or warped, which may result in the loss of some key information.

Al-Kasassbeh et al. [48], proposed a method for classifying malware based on n-grams analysis to extract important features from the executable files, such as opcodes, imports, and gray-scale images. The authors used different feature selection methods, such as principal component analysis, correlation-based feature selection, and information gain, to reduce the dimensionality and select the most relevant features, and then used different machine learning classifiers, such as Support Vector Machine, K-Nearest Neighbor, and Decision Tree to classify the files as benign or malicious. Iadarola et al. [49], while proposing a method for detecting malware families based on image analysis, have also presented a comparison of various feature extractions techniques and the ML classifiers algorithms based on the image analysis. The authors converted executable files into greyscale images, and then extracted features from the images using different methods, such as local binary patterns, histogram of oriented gradients and scale-invariant feature transform, and then used different machine learning classifiers to classify the images into malware families. The authors highlighted that their method achieved high accuracy and low false positive rate in malware detection and provided more insight into the visual patterns of malware families.

Even the usage of machine learning algorithms for detection of ransomware presents their own set of challenges. One challenge is the lack of labeled training data, which is necessary for supervised learning algorithms. This is due to the low prevalence of ransomware attacks, making it difficult to collect sufficient data to train the

model. Another challenge is the ability of ransomware to constantly evolve, making it difficult for the model to generalize to new and unknown types of ransomware. Additionally, the high dimensionality of the data used in machine learning models can result in overfitting and reduced generalization performance. In addition to these, obfuscation, anti-analysis techniques and Ransomware-As-A-Service (RAAS) also increase the complexity and difficulty of having a single solution for detecting ransomware.

Feature extraction and selection is a vital step and directly affects the efficacy of any machine learning based detection technique. As presented above, a lot of research has gone into this area and researchers have proposed various ways of handling this step effectively. One way to address these challenges is by using a meta-heuristic optimization algorithm, such as Particle Swarm Optimization algorithm. The PSO algorithm works by simulating a swarm of particles, where each particle represents a potential solution to the optimization problem. In feature selection, each particle represents a subset of features that can be selected for a given problem. The PSO algorithm iteratively updates the position and velocity of each particle in the search space, based on the best solution found so far and the social influence of the other particles [50]. In the context of feature selection, the PSO algorithm can be used to search for an optimal subset of features that maximizes a fitness function, which for a ransomware detection method, will be the accuracy of the wrapped classification algorithm. Abbasi et al. [51], proposed a 2-stage feature selection technique, where the features were divided into multiple groups and then as the first stage, used Mutual Information (MI) for feature ranking and select equal number of top features in each of the feature groups. To further improve the selection, the PSO was used in the second stage to select the optimum number of features from each group based on a wrapped Regularized Logistic Regression (RLR) classification algorithm. The same authors have done further research [52] and compared the performance of variants of PSO, such as Variable-Length Particle Swarm Optimization (VarLenPSO) and Self-adaptive Particle Swarm Optimization (SaPSO). They have also experimented with different classification algorithms, such as, Regularized Logistic Regression (RLR), Random Forest (RF), Decision Tree (DT), Support Vector Machine (SVM) and K-Nearest Neighbor (KNN) in both binary and multi-class setup, and how they affect the overall accuracy of the model. However, in this research, it can be noted that each classification algorithm performed differently in different setups. Not one classification algorithm consistently outperformed others.

As highlighted in many of the research referenced above, the selection of a machine learning model for

classification of a sample is also a crucial step after the feature extraction and selection are performed. In many cases, the researchers used a wrapper-based approach combining the feature extraction, selection and classification techniques together to get the best detection accuracy. In this approach, the efficacy of the model depends on the combined performance of the complete system of selection of algorithms and techniques. Khammas [53], compared the performance of Random Forest, Ada Boost M1, Bagging and Rotation Forest algorithms in detection of ransomware based on features extracted from static analysis of the samples and proved that Random Forest outperformed others while maintaining a low False Positive and False Negative rates. Alqahtani et al. [54], presented a survey of existing research on the classification algorithms and their performance in detecting malware targeted at Android mobile platform. In their research, they found that most of the researchers used Support Vector Machine (SVM) or Naïve Bayes (NB) classifiers. However, they found that other classifiers such as Decision Tree (DT), Simple Logistic (SL) also performed well.

Masum et al. [55], proposed a detection technique based on feature extracted from static analysis of samples while comparing the performance of various classification algorithms. They found that Random Forest (RF) algorithm consistently outperformed other algorithms, such as Decision Tree (DT), Naïve Bayes (NB), Logistic Regression (LR) as well as Neural Network (NN)-based classifiers, although Naïve Bayes maintained a better Recall score. Noorbehbahani et al. [56], also compared the performance of various classifiers in detecting malware in CICAndMal2017 dataset [57]. They identified that Random Forest was best in detecting the malware when compared against Decision Tree (DT), Random Tree (RT), K-Nearest Neighbors (KNN), Naive Bayes (NB) and SVM.

While usage of a single classification algorithm is prevalent due to less time taken by the single algorithm, more research is being conducted in studying the usage of multiple classification algorithms together, known as ensemble models, in an effort to increase the detection accuracy. As any given algorithm has both advantages and disadvantages, by combining heterogeneous algorithms using different techniques, the researchers made efforts to bring out the best in each of those classification algorithms while minimizing their shortcomings [47, 32, 46, 36, 7, 44].

Ahmed et al. [58], proposed an approach for mitigating adversarial evasion attacks of ransomware using ensemble learning. The paper presented an analysis approach to detect Android malware by monitoring memory usage and system calls and then compared the performance of various combinations of classifier algorithms in an ensemble setup. They concluded that an ensemble setup with based classifiers of SVM with Sequential Minimal Optimization

(SMO), Logistic regression, Simple Logistic Regression, AdaBoost with SVM base and Adaboosting outperformed other ensemble models.

However, selection of the algorithms can vastly affect the performance of the overall ensemble classification model. We need to identify shortcomings of each algorithm and then find a suitable algorithm which can fill in that gap, and this new combination should fit in well with the selection of features and not affect each other's performance and reduce the overall system performance. Furthermore, usage of multiple classifiers can be computationally expensive and require significant computational resources to train and evaluate and might not be suitable in real-time detection on computers with less resources.

2.4.3 Network-based detection techniques

Network-based detection works on monitoring the network traffic in an aim to detect ransomware. Most ransomware usually exhibits a characteristic where they need to communicate with a Command and Control (C&C) server to obtain the encryption keys and receive instructions on when to decrypt the data after the victim has paid the ransom. A network-based detection technique attempts to detect possible ransomware through analyzing the network traffic and possibly flag and even prevent any unknown or malicious communications. This sometimes uses a combination of existing techniques such as a Firewall to statically monitor the traffic and machine-learning models which analyze network traffic and identify patterns that are indicative of ransomware activity. This approach is particularly useful for detecting new or previously unknown ransomware variants, which may not be recognized by traditional signature-based detection methods. In addition to detecting ransomware attacks, network-based detection can also be used to prevent ransomware infections from spreading throughout a network. By isolating infected systems and preventing them from communicating with other systems on the network, security analysts can limit the damage caused by a ransomware attack and prevent it from spreading further.

Akbanov et al. [59], proposed an SDN-based approach to detect and mitigate the WannaCry ransomware. They used a combination of network flow-based and payload-based analysis to identify and block WannaCry traffic. However, this uses a static list of hosts/IP addresses typically used by WannaCry ransomware as its C&C server and then blocks any communication with those servers. This cannot detect or prevent any other type of ransomware or even a new variant of WannaCry communicating to a different host. Meanwhile, Almarshadani et al. [60], presented a multi-classifier network-based approach for detecting crypto ransomware, with a specific focus on the

Locky ransomware. They experimented with 2 types of classifiers each working on packet-level and flow-level feature set in detecting Locky ransomware. Although the research does propose a way to extract network feature vectors of a ransomware and able to achieve around 97% detection accuracy, this technique may not be effective for detecting other types of ransomware attacks.

Cabaj et al. [61], proposed a software-defined networking (SDN)-based approach to detect crypto ransomware by analyzing HTTP traffic characteristics. They used features such as header size, payload size, and content type to train their system achieving a detection accuracy of 97-98%. However, their proposed detection method was only trained and tested on Locky and CryptoWall ransomware. Similarly, Cusack et al. [62], proposed an SDN-based approach to detect ransomware using machine learning. They used a combination of SDN and Random Forest machine learning algorithm to detect ransomware by classifying the stream of HTTPs communications between the ransomware and their C&C servers. The proposed approach achieved an average detection accuracy of 87%.

Alhawi et al. [63], proposed a machine learning-based approach to detect and classify ransomware network traffic on Windows platforms. They used various machine learning algorithms such as Random Forest, Decision Tree, K-Nearest Neighbor, and Support Vector Machine to train their system. The proposed approach achieved an average detection accuracy of around 97%. Morato et al. [64], proposed a method for detecting ransomware by analyzing file sharing traffic. They used a combination of machine learning and network flow-based analysis to detect ransomware at the early stage of the infection. The proposed approach achieved a detection accuracy of 99.9% in the early stage of the attack. Wan et al. [65], also proposed Intrusion Detection System (IDS) which is based on results provided by Decision Tree classification algorithm. They collected and extracted features from the network traffic of ransomware and benign samples and used them to train and test Decision Tree classification algorithm. The results or features found to be important for classification are then used as rules in an IDS in an attempt to quick and early detection of ransomware.

2.4.4 Hybrid detection techniques

Hybrid techniques employ multiple detection and protection techniques in a multi-layered approach for detection and prevention of ransomware attacks. Jr and Elleithy [66], proposed a five-layered approach to prevent and safeguard against ransomware attacks, which includes perimeter defense, network defense, endpoint defense, data backup and recovery, and end-user awareness and training. Patyal et al. [67], proposed a multi-layered defense

architecture against ransomware attacks consisting of six layers: network, host, application, data, backup, and user education. They discussed the challenges of preventing and recovering from ransomware attacks and argued that traditional security solutions are ineffective against such attacks. They also emphasized the importance of regularly testing and updating the defense architecture to ensure its effectiveness.

2.5 Conclusion

Recent research on ransomware has proposed various prevention, detection, and recovery techniques, including user education, network segmentation, software patching, antivirus software, intrusion detection systems, and machine learning-based approaches. Machine learning algorithms have been used for identifying ransomware propagation mechanisms, encryption patterns, and malicious network traffic. Machine learning algorithms have also been used in feature selection problem space to select the best set of features in classifying the ransomware. Lastly, the challenges within each of these techniques have been highlighted and discussed on how other techniques have tried to overcome those challenges or at least minimize the effects of those.

3 Research Methodology

The research experimentation was divided into multiple stages, each tailored to specific tasks and their corresponding outcomes.

3.1 Data Collection

The ransomware samples utilized in this study were obtained from MalwareBazaar¹, an online repository hosting diverse types of malware along with their associated metadata. To establish the dataset for this research, a Python script was utilized to extract a list of 1000 most recent ransomware samples obtained through this platform. These samples were grouped by the ransomware families they belong to and then the families containing fewer than 5 executables were excluded from the list to ensure a more comprehensive selection. Additionally, the list was further refined to include a multiple of 5 ransomware instances from each family. This was carried out to facilitate the division of the dataset into an 80% training subset and a 20% testing subset. As a result, a total of 175 ransomware executables from 28 distinct

¹ <https://bazaar.abuse.ch/api>

Table 1 Ransomware families analyzed in this research

Ransomware Family	# of samples
Azov	5
Babuk	5
Blackbasta	10
Blackcat	5
Blackcocaine	5
Blackmatter	5
Chaos	10
Conti	5
Cyborg	5
Dharma	10
Gandcrab	5
GlobeImposter	5
Hiddentear	5
Hive	5
Lockbit	10
Makop	10
Medusalocker	5
Neshta	5
Nitro	5
Njrat	5
Phobos	5
Ransomware	5
RedlineStealer	5
Seven	5
Stop	5
Thanos	5
WannaCry	10
Xorist	10

ransomware families were obtained for analysis. The distribution of these samples is presented in Table 1

A dataset comprising of benign and malware samples, compiled by Tuan et al.[68], was utilized to collect a subset of 45 benign samples. Among these samples, 40 were designated for training purposes, while the remaining 5 were allocated as test samples. The collected samples encompass a diverse range of portable applications and utilities commonly used on the Windows operating system.

3.2 Simulation environment

As this research aims to study the static features of ransomware as well as dynamic features, we need to execute the collected samples in a controlled environment,

collect the logs and then extract relevant static and dynamic features that can be used to distinguish between ransomware and benign nature of the sample. The PE Parser, a Python library, can be utilized to extract static features [69] while dynamic features can be obtained by utilizing sandboxing tools like Cuckoo² [70] or DRAKVUF³ [71]. We chose DRAKVUF as Cuckoo requires an agent to be running on the guest machine, whereas DRAKVUF does not, as DRAKVUF monitors the application's behavior through Virtual Machine Introspection (VMI) technology, which happens by directly connecting to the hypervisor [72]. This prevents ransomware from detecting sandbox environment and changing its behavior, which leads to a more accurate representation of real-world ransomware behavior.

DRAKVUF can only be installed on Ubuntu Operation System (OS). A Virtual Machine (VM) with a 4 core CPU and 16GB RAM was created to host Ubuntu 20.04 LTS (Focal Fossa) and then DRAKVUF was installed on top of it. And to run the ransomware, the sandboxed nested VM was running the Windows 10 OS and has the configuration of 2 core CPU and 4GB of RAM. As part of this setup, a collection of documents [73] was placed in one of the folders within the VM. These documents were intentionally chosen to act as the target files that could potentially be encrypted by the ransomware.

3.3 Feature extraction

DRAKVUF was configured to monitor only system API calls originating from the sample's execution. This was done to limit the scope of monitoring and to reduce the overall log file size.

Algorithm 1: Executing samples in the sandbox

Input: samplesFolder

Output: Dataset in CSV format

```

1 begin
2   Load sandboxVMImage to create a baseline state of the
   sandbox environment
3   foreach sampleFile in samplesFolder do
4     Restore the sandbox VM to the baseline state
5     Copy sampleFile into the sandbox VM
6     Start monitoring the API calls using DRAKVUF
7     RunSample(sampleFile)
8     Stop monitoring the API calls and collect the logs

```

To automate the execution of samples, a Python script was utilized as presented in Algorithm 1. This script reads the list of benign and ransomware samples from a

² <https://cuckoosandbox.org>

³ <https://drakvuf.com>

designated folder and triggers DRAKVUF to execute each sample within the sandboxed virtual machine (VM) environment. By utilizing DRAKVUF's command line options, the script redirects the logs capturing the sample's behavior to a separate file in another designated folder. Each sample's behavior is monitored for 1 minute. The sandbox will then be destroyed, and a new sandbox will be created from the backup to execute another ransomware or benign sample.

3.4 Feature Engineering

A separate Python script was employed to extract the count of each system call made by the sample application as presented in Algorithm 2. Within the same script, PE Parser, a Python library, is utilized to extract the static features of the samples. These static features encompass sample metadata features, 1-Gram Opcode features, registry references, data defined directives, symbol frequencies, PE section features, and other miscellaneous features. The number of features in each category is given in Table 2.

Algorithm 2: Extract features and prepare dataset

Input: drakvufLogs, samplesFolder

Output: Dataset in CSV format

```

1 begin
2   foreach sampleFile in samplesFolder do
3     Read the corresponding DRAKVUF analysis log
4     Count number of occurrences of each API Call in a
      Python Dictionary
5     Use PE Parser to obtain static features
6     Convert the static and dynamic features count to a
      pandas DataFrame
7     Append it to the extractedFeatures
8     Append the sample label to the extractedFeatures
9   Saves the extracted features into a CSV file

```

Following the extraction of these features, the Python script generates a dataset file intended for use by machine learning models. Each cell within this dataset contains a count representing the occurrence of a specific feature extracted from the sample. The final column in the dataset file contains the classification or label for each sample. Ultimately, a total of 105 static features and 260 dynamic features were extracted from each sample.

3.5 Model implementation and experiments

The Particle Swarm Optimization (PSO) algorithm was implemented in this research by directly integrating it from the PySwarms Python library [74]. The cognitive parameter

Table 2 Categories of collected features and their count

Feature category	Number of features
Metadata	2
Symbols frequency	7
1-gram opcode	22
Registry	20
Section	5
Data define directives	5
Miscellaneous	44
Dynamic API Calls	260

(c1) and social parameter (c2) were set as 1.25, the inertia parameter (w) as 0.2 and the swarm size was set as 5. In our implementation, PSO was used to calculate the fitness value of each feature and their impact on the overall detection accuracy. Consequently, these fitness values calculated by the PSO algorithm were multiplied by their corresponding features to obtain the final feature selection.

As PSO uses wrapped classification algorithm for evaluating the performance of the swarm of features, this steps also includes implementation and usage of the proposed ensemble-based machine learning model by combining RF – SVM and DT-KNN. For these classification algorithms, the scikit-learn Python library [75] was utilized. To determine the optimal hyperparameters for all the classification algorithms, a grid search technique was employed. The search determined that setting the number of trees to 30 and the maximum number of features to consider before splitting a node to the square root of the total number of features yielded the best results for the RF classifier. Additionally, based on the distribution of our dataset and after splitting it into an 80-20 ratio, a multiple of 4 samples were selected as the training dataset, leading to the decision of setting the minimum number of sample leaves to 4. For the SVM classifier, the grid search validated that setting the C parameter to 100 and the gamma parameter to 0.001, along with the default Radial Basis Function (RBF) kernel function, was the most suitable choice for our problem. Grid search was also performed for tuning the hyperparameters of Decision Tree (DT) and K-Nearest Neighbors (KNN). For DT, the criterion used to measure the quality of a split was selected as Gini impurity. The maximum depth of the tree was set to 5, and the minimum number of samples required to be at a leaf node was set to 5. For KNN, the number of neighbors was set to 4, the power parameter for the Minkowski metric, which determines the distance between points, was set to 1. Additionally, the weights function for KNN was chosen as the inverse of the distance between neighbors. This means that closer neighbors to a query point would have a greater

influence on the classification decision compared to neighbors that are further away. Through our experiments, this particular setup was found to be suitable enough to validate our proposed approach.

Finally a Voting Classifier technique was employed to integrate the outcomes from each individual classifier. In this approach, soft voting was used, considering the probabilities generated by each classifier. For the RF and SVM ensemble, weights of 5 and 4 were assigned respectively. This means that the probability calculated by the RF classifier had 1.25 times more influence than the probability from the SVM classifier. Similarly, for the KNN and DT ensemble, weights of 5 and 4 were utilized. These weight values were determined through multiple experiments, aiming to achieve the highest detection rate while minimizing the occurrence of False Negatives and False Positives.

The experiments were conducted using the specified parameters, running them independently 100 times. Each run consisted of 100 epochs. After training the various approaches using the training dataset, the same models were evaluated using the test data. To obtain the final result for each approach, the average of all iterations was calculated, considering it as the representative performance for that specific approach. Furthermore, the individual classifiers were assessed independently using the test data in order to gain a comprehensive understanding of their individual performance when compared against the ensemble classifiers. Additionally, a similar arrangement of classifiers were trained and tested without incorporating PSO for feature selection. As a result, in this setup, all features in the dataset were assigned equal weights without any bias. This experiment was conducted to assess the impact of utilizing PSO for feature selection on the detection rate and to validate our proposed approach.

The code implementation for this is available on GitHub⁴. It includes the implementation of various machine learning models, the training and test datasets, as well as the results obtained from these models.

4 Results and discussion

After running the experiments, the results were captured in different formats to be able to evaluate the performance of classifier setups in various measurement criteria. The results were measured in terms of accuracy, precision, recall and F1-score rates. Furthermore, we will also discuss features identified by PSO as most influential in classifying the samples.

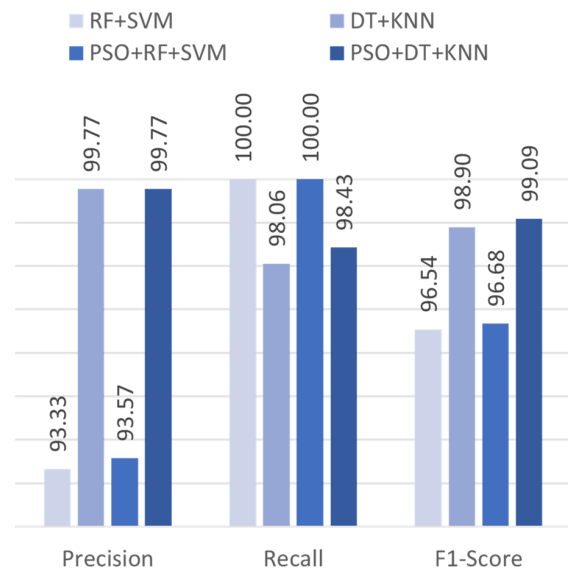
Table 3 Classification accuracy of all methods

Classifier setup	Detection accuracy in %		
	PSO for selection	feature	All equal weighted features
Ensemble RF+SVM	93.82		93.56
Ensemble DT+KNN	98.38		98.05
RF	93.05		93.00
SVM	89.74		89.74
DT	94.13		93.36
KNN	98.36		97.44

4.1 Accuracy

The detection rates when evaluated against the test dataset are presented in Table 3. The best detection rates among the various methods are highlighted in bold.

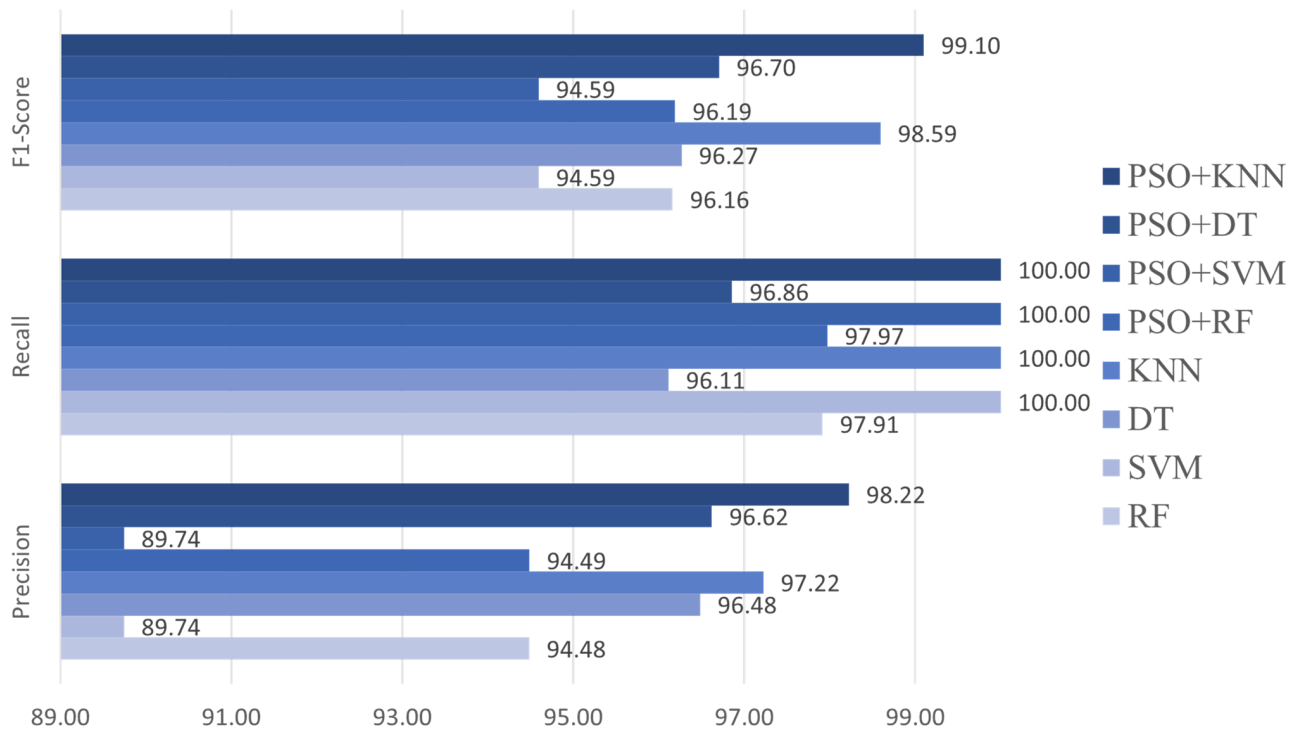
Fig. 1 Performance of ensemble classifiers



It is worth mentioning that when the Decision Tree and K-Nearest Neighbors classifiers were combined in an ensemble classifier setup using weighted features selected by PSO, the highest accuracy rate of 98.38% was achieved. On the other hand, when the DT and KNN combination was used with all features without any weights, the accuracy rate was slightly lower at 98.05%.

When comparing the accuracy rates of the ensemble setups using Random Forest and Support Vector Machine, the accuracy rate was 93.82% when weighted features were

⁴ <https://github.com/neelkumarg/PSOwithEnsembleClassifiers>

Fig. 2 Performance of individual classifiers

used. In contrast, the accuracy rate decreased slightly to 93.56% when unweighted features were utilized.

When examining the individual classifiers, RF with unweighted features performed slightly better than its counterpart with weighted features. DT and KNN on the other hand showed significant improvements when used with weighted features. However, no difference in performance was observed for SVM when using weighted or unweighted features.

In terms of the comparison between the individual and combined ensemble classifier models, notable differences were observed. These differences were even significant in the case of the unweighted features set. For instance, the DT and KNN ensemble setup achieved an accuracy rate of 98.05%, whereas individually, they only achieved 93.36% and 97.44%, respectively.

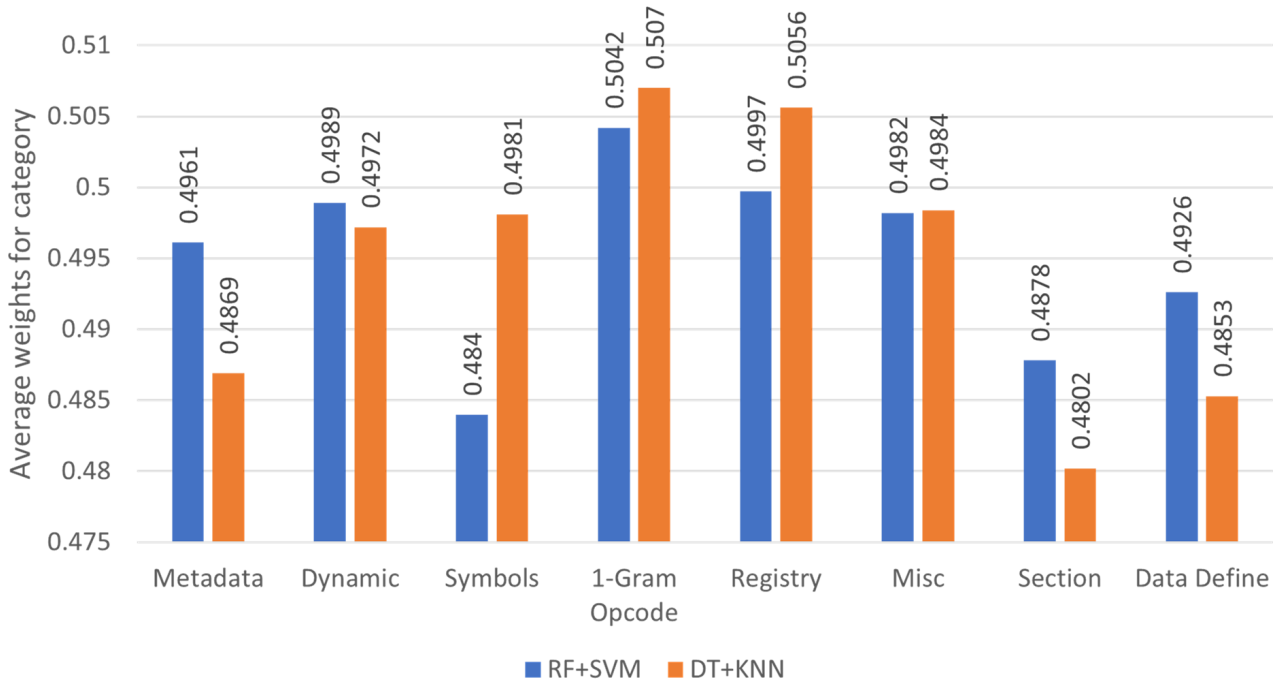
Although the accuracy rate is a valuable metric for evaluating classifier performance, it is essential to consider other measures as well. Evaluating the number of False Positives and False Negatives is crucial. Precision, Recall, and F1-Score can be calculated to assess these aspects, providing a more comprehensive evaluation of classifier performance. By examining these measures, a more comprehensive evaluation of the classifiers can be obtained beyond accuracy alone.

4.2 Precision, Recall and F1-Score

Precision, a critical metric for evaluating classifiers, measures the accuracy of positive predictions. As presented in Figure 1, among the ensemble classifiers, the DT+KNN model achieved a precision score of 99.77% regardless of its integration with PSO, while the RF+SVM combination obtained 93.57% with PSO and 93.33% without PSO. These results indicate successful identification of positive instances.

Recall, or sensitivity, evaluates a classifier's ability to detect positive instances. In both ensemble classifiers, RF+SVM achieved a perfect recall rate of 100%, regardless of the integration of PSO. This signifies the classifier's high proficiency in capturing positive instances. On the other hand, DT+KNN achieved recall rates of 98.43% and 98.06% when integrated with PSO and when using unweighted features, respectively.

The F1-score provides a comprehensive assessment of a classifier's performance by considering both precision and recall. In this context, the ensemble classifiers demonstrated similar performances, with DT+KNN utilizing weighted features achieving the highest F1-score of 99.09%, demonstrating a well-balanced performance between precision and recall. Conversely, the RF+SVM ensemble classifier using unweighted features achieved the lowest F1-score of 96.54%.

Fig. 3 Average fitness values for each feature category

Overall, the precision, recall, and F1-score metrics highlight the effectiveness of the ensemble classifiers. DT+KNN consistently demonstrated strong precision and F1-scores, while RF+SVM exhibited competitive performance, especially when integrated with PSO.

In Figure 2, among the single classifiers integrated with PSO, KNN exhibited a precision rate of 98.22%, surpassing other individual classifiers. Without integrating PSO, KNN achieved a precision rate of 97.22%, demonstrating its strong performance. It's noteworthy that precision of KNN improved by 1% when it was paired with PSO. In contrast, SVM had the lowest precision rate of 89.74%, regardless of its integration with PSO.

On the other hand, KNN and SVM achieved perfect recall scores irrespective of PSO integration, indicating their ability to detect positive instances effectively. This highlights that SVM has classified a marginally higher number of benign samples as malicious leading to False Positives. The other classifiers generally performed similarly with or without PSO integration.

In terms of F1-score, KNN consistently achieved the best overall performance, scoring 99.10% with PSO integration and 98.59% without it. The performance of DT also showed an improvement of 0.43% when it was integrated with PSO. Other classifiers showed comparable performances regardless of PSO integration.

These findings emphasize the varying performance of different classifiers in terms of precision, recall, and F1-

score. KNN stood out with its high precision and varying recall rates, while KNN consistently demonstrated strong overall performance. The integration of PSO influenced the performance of some classifiers, particularly KNN. On the other hand, precision and recall rates of SVM point to a greater number of False Positives.

4.3 Fitness values of the features by PSO

Creating a plot to visualize the fitness values or weights assigned to each feature in the dataset, as calculated by PSO when combined with RF+SVM and DT+KNN, offers a more comprehensive perspective on the features. Understanding the weights calculated by Particle Swarm Optimization (PSO) in a model is important for various reasons. Firstly, the weights indicate the relative importance of features, allowing for identification of key features in the model's decision-making process. Secondly, they provide insights into the model's inner workings, helping explain predictions and address potential biases. Additionally, analyzing the weights can assess the effectiveness of the optimization process and guide further improvements in model performance. Moreover, feature weights aid in feature selection, simplifying the model and potentially improving generalization.

In both the RF+SVM and DT+KNN models, the weights calculated by PSO for different feature categories are relatively close to each other.

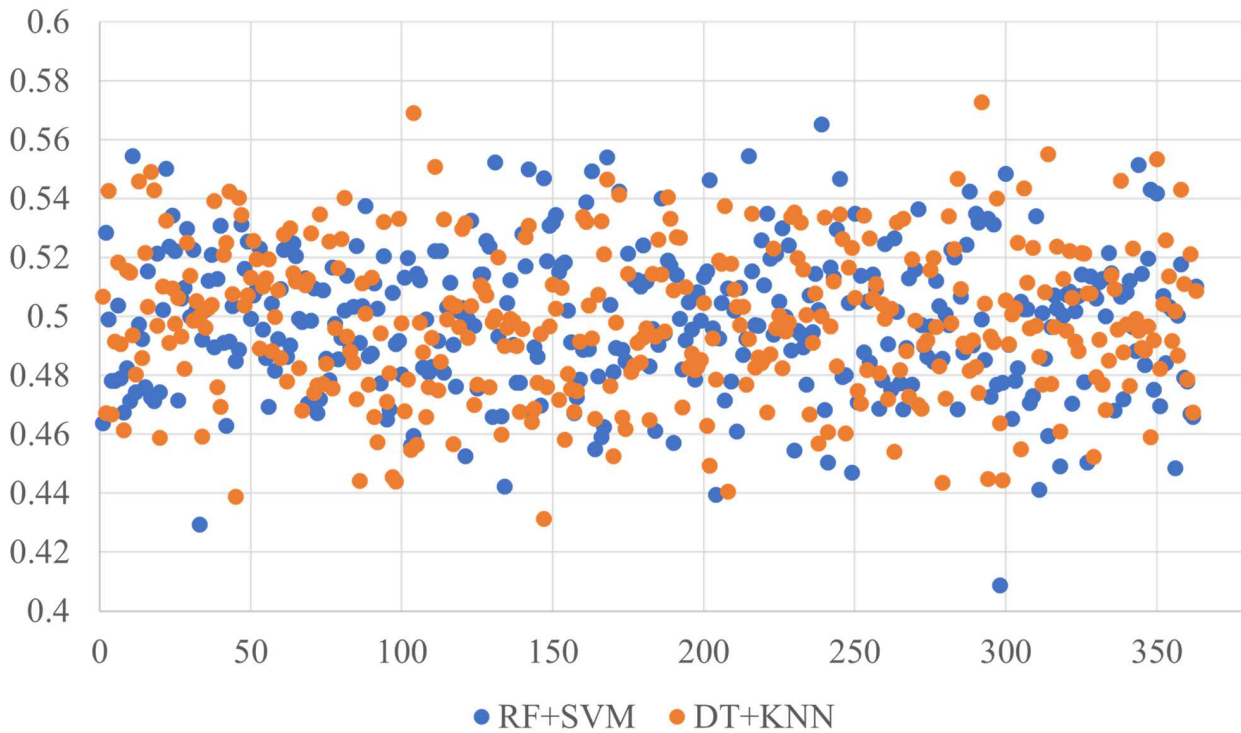
Fig. 4 Fitness values of features calculated by PSO

Figure 3 presents the weights distribution assigned by PSO for various categories of features. For the RF+SVM model, the highest weight is assigned to the "1-Gram Opcode" category with a value of 0.5042, indicating its significance in determining ransomware. The other feature categories, such as "Metadata," "Dynamic," "Symbols," "Registry," "Misc," "Section," and "Data Define," also have weights ranging from 0.484 to 0.4997, suggesting their comparable importance in the model. Similarly, in the DT+KNN model, the "1-Gram Opcode" category holds the highest weight of 0.507, reinforcing its importance in identifying ransomware. The other feature categories, including "Metadata," "Dynamic," "Symbols," "Registry," "Misc," "Section," and "Data Define," have weights ranging from 0.4802 to 0.5056, indicating their relatively balanced contribution to the model.

Overall, these observations suggest that multiple feature categories, both static and dynamic, play a role in detecting ransomware, as indicated by their comparable weights in both models. The "1-Gram Opcode" category consistently stands out as a significant feature in both models, emphasizing the importance of assembly language instructions in ransomware detection.

Examining Figure 4 reveals that the weights assigned to the features fall predominantly within the range of 0.44 to 0.56, indicating a relatively balanced importance among the collected features for the model's performance. This suggests

Table 4 Top 10 features identified by PSO when combined with the ensemble models

RF+SVM	DT+KNN
ASM_1GRAM_OPCODES_al	ASM_1GRAM_OPCODES_call
ASM_1GRAM_OPCODES_jg	ASM_1GRAM_OPCODES_dw
NtAlpcSendWaitReceivePort	ASM_DD_dc_por
NtTraceControl	NtSetEvent
NtOpenProcessToken	NtCreateFile
NtCreateFile	NtQueryFullAttributesFile
NtUnsubscribeWnfStateChangeNtMapViewOfSectionEx	
NtCreateWnfStateName	NtGetCachedSigningLevel
NtAcceptConnectPort	NtReadFileScatter
NtQueryDefaultUILanguage	NtUnloadKeyEx

that the feature extraction method employed captured a set of features that are generally equally significant for both the RF+SVM and DT+KNN models.

Based on the Table 4, it appears that the RF+SVM and DT+KNN models have identified different features as being important for detecting ransomware. In the RF+SVM model, the features ASM_1GRAM_OPCODES_al and ASM_1GRAM_OPCODES_jg are highlighted as influential. These features represent specific assembly language instructions related to addition and jump-if-greater

operations, respectively. Additionally, the RF+SVM model considers features such as `NtAlpcSendWaitReceivePort`, `NtTraceControl`, `NtOpenProcessToken`, `NtCreateFile` and others as significant for detecting ransomware.

On the other hand, the DT+KNN model identifies `ASM_1GRAM_OPCODES_call` and `ASM_1GRAM_OPCODES_dw` as the most influential features. These features represent assembly language instructions related to "call" and "Define Word" operations, respectively. The DT+KNN model also considers features like `NtSetEvent`, `NtQueryFullAttributesFile`, `NtMapViewOfSectionEx`, `NtGetCachedSigningLevel`, and others as important indicators of ransomware.

In summary, the observations indicate that different combinations of features, including assembly language instructions and specific Windows API calls, are deemed important by the RF+SVM and DT+KNN models in identifying ransomware.

5 Conclusion

In this study, we proposed a novel approach that combined ensemble classifiers with feature selection using the Particle Swarm Optimization (PSO) algorithm. Two separate ensemble models were created, one consisting of Random Forest (RF) and Support Vector Machine (SVM) classifiers, and the other consisting of Decision Tree (DT) and K-Nearest Neighbors (KNN) classifiers. The PSO algorithm was employed to determine the optimal features and their corresponding fitness values in each ensemble classifier.

The results obtained from the experiments demonstrated that integrating PSO for feature selection improved the overall detection rate compared to using all features with equal weights. By identifying the most relevant features and assigning appropriate fitness values, the ensemble classifiers achieved higher accuracy, reduced false positives, and false negatives. Additionally, the individual classifiers were evaluated separately, providing insights into their individual performance, and further reinforcing the effectiveness of the ensemble models.

While this research successfully explored the integration of PSO for feature selection with ensemble classifiers, there are several avenues for future investigation and improvement. Some potential future directions include:

- Exploration of alternative feature selection techniques: Although PSO proved effective in this study, other feature selection algorithms, such as Genetic Algorithms or Ant Colony Optimization, could be investigated to compare their performance and identify the most suitable technique for specific datasets.
- Investigation of additional ensemble models: While this research focused on the RF-SVM and DT-KNN ensemble models, there are various other combinations that could be explored, such as combining boosting algorithms with neural networks or utilizing multiple instance learning methods.
- Incorporation of additional classifiers: Including a wider range of classifiers in the ensemble models may further enhance the overall performance. Techniques such as stacking or cascading could be employed to combine diverse classifiers and exploit their individual strengths.
- Evaluation on different datasets: The proposed approach should be tested on various datasets from different domains to assess its generalizability and determine its effectiveness in diverse real-world scenarios.
- Optimization of hyperparameters: Fine-tuning the hyperparameters of the individual classifiers and the ensemble models could potentially improve their performance. Techniques like grid search or Bayesian optimization can be applied to determine the optimal values for these hyperparameters.

In conclusion, the integration of PSO for feature selection with ensemble classifiers showcased promising results in this study. Further research and enhancements in the aforementioned areas can contribute to the development of more robust and accurate models for various classification tasks.

6 Compliance with Ethical Standards

The entirety of this research, including expenses related to data collection, data processing, software tools, and hardware, has been fully financed by the authors themselves, and no external funding has been sought or obtained for these endeavors. All experiments involving software samples and subsequent analyses were conducted solely for research purposes within controlled environments. Data privacy and security were meticulously upheld by using publicly available software samples and safeguarding against the inclusion of any sensitive or personal information in the dataset or analysis logs. The execution of ransomware and benign samples took place exclusively within isolated sandbox environments, preventing any potential external harm and minimizing unintended consequences. As no human participants were involved, informed consent was not applicable, although ethical deliberations surrounding the analysis were taken into account to ensure responsible research practices. The methodology's transparency is a fundamental aspect of this study, with complete procedures, algorithms, and steps described in detail to facilitate reproducibility and enable

comprehensive evaluation. The authors have conscientiously attributed all tools, libraries, and datasets used in the study and invite constructive critique and peer review to enhance the integrity of the research.

This paper maintains a strong commitment to ethical standards throughout the research process. All activities have been carried out with integrity and in compliance with established guidelines to ensure the credibility and responsibility of the findings presented.

7 Competing Interests

The authors declare that there are no conflicts of interest associated with the conception, execution, or publication of this research. The study was undertaken with complete impartiality and objectivity, and the authors have no financial, professional, or personal affiliations that could potentially bias or influence the presentation of the study's findings.

8 Research Data Policy and Data Availability Statements

The datasets generated during and/or analysed during the current study are available in the GitHub repository, <https://github.com/neelkumarg/PSOwithEnsembleClassifiers>. The software and ransomware samples used in this study were sourced from publicly accessible repositories, which are duly cited within this paper.

References

- [1] Sumaiah Algarni. "Cybersecurity Attacks: Analysis of "WannaCry" Attack and Proposing Methods for Reducing or Preventing Such Attacks in Future". In: *ICT Systems and Sustainability* (2021). Ed. by Milan Tuba, Shyam Akashe, and Amit Joshi, pp. 763–770. doi: http://dx.doi.org/10.1007/978-981-15-8289-9_73.
- [2] Collin Eaton and Dustin Volz. "Colonial Pipeline CEO tells why he paid hackers a 4.4 million ransom". In: *The Wall Street Journal* (2021). URL: <https://www.wsj.com/articles/colonial-pipeline-ceo-tells-why-he-paid-hackers-a-4-4-million-ransom-11621435636>.
- [3] Tsvetan Tsvetanov and Srishti Slaria. "The effect of the Colonial Pipeline shutdown on gasoline prices". In: *Economics Letters* 209 (2021), p. 110122. issn: 0165-1765. doi: <https://doi.org/10.1016/j.econlet.2021.110122>. URL: <https://www.sciencedirect.com/science/article/pii/S0165176521003992>.
- [4] Ali Ahmed Mohammed Ali Alwashali, Nor Azlina Abd Rahman, and Noris Ismail. "A Survey of Ransomware as a Service (RaaS) and Methods to Mitigate the Attack". In: *2021 14th International Conference on Developments in eSystems Engineering (DeSE)*. 2021, pp. 92–96. doi: [10.1109/DeSE54285.2021.9719456](https://doi.org/10.1109/DeSE54285.2021.9719456).
- [5] Md Jobair Hossain Faruk et al. "Malware Detection and Prevention using Artificial Intelligence Techniques". In: *2021 IEEE International Conference on Big Data (Big Data)*. 2021, pp. 5369–5377. doi: [10.1109/BigData52589.2021.9671434](https://doi.org/10.1109/BigData52589.2021.9671434).
- [6] Nahush Shetty and Raja Praveen. "A survey paper on malware detection techniques". In: *International Journal* 10 (2 2021). doi: <https://doi.org/10.30534/ijatcse/2021/151022021>. URL: <http://www.warse.org/IJATCSE/static/pdf/file/ijatcse151022021.pdf>.
- [7] Abdullah Alqahtani and Frederick T. Sheldon. "A Survey of Crypto Ransomware Attack Detection Methodologies: An Evolving Outlook". In: *Sensors* 2022, Vol. 22, Page 1837 22 (5 Feb. 2022), p. 1837. issn: 1424-8220. doi: [10.3390/S22051837](https://doi.org/10.3390/S22051837). URL: <https://www.mdpi.com/1424-8220/22/5/1837>.
- [8] Routa Moussaileb, Renzo E. Navas, and Nora Cuppens. "Watch out! Doxware on the way..." In: *Journal of Information Security and Applications* 55 (Dec. 2020), p. 102668. issn: 2214-2126. doi: [10.1016/J.JISA.2020.102668](https://doi.org/10.1016/J.JISA.2020.102668).
- [9] Per Håkon Meland, Yara Fareed Fahmy Bayoumy, and Guttorm Sindre. "The Ransomware-as-a-Service economy within the darknet". In: *Computers & Security* 92 (May 2020), p. 101762. issn: 0167-4048. doi: [10.1016/J.COSE.2020.101762](https://doi.org/10.1016/J.COSE.2020.101762).
- [10] Hiroki Kuzuno, Shun Inagaki, and Kenichi Magata. "Constructing a Complete Timeline of a Security Incident by Aggregating Reports". In: *2018 13th Asia Joint Conference on Information Security (AsiaJCIS)*. 2018, pp. 109–115. doi: [10.1109/AsiaJCIS.2018.00026](https://doi.org/10.1109/AsiaJCIS.2018.00026).
- [11] M Satheesh Kumar, Jalel Ben-Othman, and K G Srinivasagan. "An Investigation on Wannacry Ransomware and its Detection". In: *2018 IEEE Symposium on Computers and Communications*

- (ISCC). 2018, pp. 1–6. DOI: 10.1109/ISCC.2018.8538354.
- [12] S Ghafur et al. “A retrospective impact analysis of the WannaCry cyberattack on the NHS”. In: *npj Digital Medicine* 2 (1 2019), p. 98. ISSN: 2398-6352. DOI: 10.1038/s41746-019-0161-6. URL: <https://doi.org/10.1038/s41746-019-0161-6>.
- [13] Sivaraju Kuraku and Dinesh Kalla. “Emotet malware—a banking credentials stealer”. In: *IOSR Journal of Computer Engineering* (2020). URL: <https://www.iosrjournals.org/iosr-jce/papers/Vol22-issue4/Series-2/F2204023141.pdf>.
- [14] Gavin Hull, Henna John, and Budi Arief. “Ransomware deployment methods and analysis: views from a predictive model and human responses”. In: *Crime Science* 8 (1 2019), p. 2. ISSN: 2193-7680. DOI: 10.1186/s40163-019-0097-9. URL: <https://doi.org/10.1186/s40163-019-0097-9>.
- [15] Jerome Dangu. “Uncovering 2017’s largest malvertising operation”. In: *Medium* (2018). URL: <https://blog.confiant.com/uncovering-2017s-largest-malvertising-operation-b84cd38d6b85>.
- [16] Chaoying Liu and Joseph C Chen. “Malvertising Campaign Abuses Google’s DoubleClick to Deliver Cryptocurrency Miners”. In: *TrendMicro, Jan 26* (2018). URL: https://www.trendmicro.com/en_in/research/18/a/malvertising-campaign-abuses-googles-doubleclick-to-deliver-cryptocurrency-miners.html.
- [17] Aritz Arrate et al. “Malvertising in Facebook: Analysis, Quantification and Solution”. In: *Electronics* 9 (8 2020). ISSN: 2079-9292. DOI: 10.3390/electronics9081332. URL: <https://www.mdpi.com/2079-9292/9/8/1332>.
- [18] Jagsir Singh and Jaswinder Singh. “A survey on machine learning-based malware detection in executable files”. In: *Journal of Systems Architecture* 112 (Jan. 2021), p. 101861. ISSN: 1383-7621. DOI: 10.1016/J.SYSARC.2020.101861.
- [19] *Microsoft Security Bulletin MS17-010 - Critical — Microsoft Learn*. URL: <https://learn.microsoft.com/en-us/security-updates/securitybulletins/2017/ms17-010>.
- [20] Maxat Akbanov, Vassilios G. Vassilakis, and Michael D. Logothetis. “WannaCry ransomware: Analysis of infection, persistence, recovery prevention and propagation mechanisms”. In: *Journal of Telecommunications and Information Technology* (2019). URL: <http://dx.doi.org/10.26636/jtit.2019.130218>.
- [21] Mordechai Guri. “USBCulprit: USB-borne Air-Gap Malware”. In: *ACM International Conference Proceeding Series* (Nov. 2021), pp. 7–13. DOI: 10.1145/3487405.3487412. URL: <https://dl.acm.org/doi/10.1145/3487405.3487412>.
- [22] Arash Mahboubi, Seyit Camtepe, and Keyvan Ansari. “Stochastic modeling of IoT Botnet spread: A short survey on mobile malware spread modeling”. In: *IEEE Access* (2020). ISSN: 21693536. DOI: 10.1109/ACCESS.2020.3044277.
- [23] Xiaolu Zhang et al. “IoT Botnet Forensics: A Comprehensive Digital Forensic Case Study on Mirai Botnet Servers”. In: *Forensic Science International: Digital Investigation* 32 (Apr. 2020), p. 300926. ISSN: 2666-2817. DOI: 10.1016/J.FSIDI.2020.300926.
- [24] Mihail Anghel and Andrei Racautanu. “A note on different types of ransomware attacks”. In: (2019). URL: <https://eprint.iacr.org/2019/605>.
- [25] Aaron Zimba et al. “On the Economic Impact of Crypto-ransomware Attacks: The State of the Art on Enterprise Systems”. In: *European Journal for Security Research* 2019 4:1 4 (1 Jan. 2019), pp. 3–31. ISSN: 2365-1695. DOI: 10.1007/s41125-019-00039-8. URL: <https://link.springer.com/article/10.1007/s41125-019-00039-8>.
- [26] V Sai Sathyanarayan, Pankaj Kohli, and Bezawada Bruhadeshwar. “Signature Generation and Detection of Malware Families”. In: *Information Security and Privacy*. Ed. by Yi Mu, Willy Susilo, and Jennifer Seberry. Springer Berlin Heidelberg, 2008, pp. 336–349. ISBN: 978-3-540-70500-0. DOI: http://dx.doi.org/10.1007/978-3-540-70500-0_25.
- [27] Parvez Faruki et al. “AndroSimilar: Robust Statistical Feature Signature for Android Malware Detection”. In: *Proceedings of the 6th International Conference on Security of Information and Networks*. Association for Computing Machinery, 2013, pp. 152–159. ISBN: 9781450324984. DOI: 10.1145/2523514.2523539. URL: <https://doi.org/10.1145/2523514.2523539>.
- [28] Zhi-Guo Chen et al. “Automatic Ransomware Detection and Analysis Based on Dynamic API Calls Flow Graph”. In: *Proceedings of the International Conference on Research in Adaptive and Convergent Systems*. Association for Computing Machinery, 2017, pp. 196–201. ISBN: 9781450350273. DOI: 10.1145/3129676.3129704. URL: <https://doi.org/10.1145/3129676.3129704>.
- [29] Ömer Aslan Aslan and Refik Samet. “A Comprehensive Review on Malware Detection

- Approaches". In: *IEEE Access* 8 (2020), pp. 6249–6271. DOI: 10.1109/ACCESS.2019.2963724.
- [30] Routa Moussaileb et al. "A Survey on Windows-Based Ransomware Taxonomy and Detection Mechanisms". In: *ACM Comput. Surv.* 54 (6 July 2021). ISSN: 0360-0300. DOI: 10.1145/3453153. URL: <https://doi.org/10.1145/3453153>.
- [31] Faranak Abri et al. "Can Machine/Deep Learning Classifiers Detect Zero-Day Malware with High Accuracy?" In: *2019 IEEE International Conference on Big Data (Big Data)*. 2019, pp. 3252–3259. DOI: 10.1109/BigData47090.2019.9006514.
- [32] Kiran Radhakrishnan, Rajeev R Menon, and Hiran V Nath. "A survey of zero-day malware attacks and its detection methodology". In: *TENCON 2019 - 2019 IEEE Region 10 Conference (TENCON)*. 2019, pp. 533–539. DOI: 10.1109/TENCON.2019.8929620.
- [33] Zhangying He et al. "When Machine Learning Meets Hardware Cybersecurity: Delving into Accurate Zero-Day Malware Detection". In: *2021 22nd International Symposium on Quality Electronic Design (ISQED)*. 2021, pp. 85–90. DOI: 10.1109/ISQED51717.2021.9424330.
- [34] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. "Survey of machine learning techniques for malware analysis". In: *Computers & Security* 81 (Mar. 2019), pp. 123–147. ISSN: 0167-4048. DOI: 10.1016/J.COSE.2018.11.001.
- [35] Subash Poudyal, Kul Prasad Subedi, and Dipankar Dasgupta. "A Framework for Analyzing Ransomware using Machine Learning". In: *Proceedings of the 2018 IEEE Symposium Series on Computational Intelligence, SSCI 2018* (Jan. 2019), pp. 1692–1699. DOI: 10.1109/SSCI.2018.8628743.
- [36] Nureni Ayofe Azeez et al. "Windows PE Malware Detection Using Ensemble Learning". In: *Informatics 2021, Vol. 8, Page 10* 8 (1 Feb. 2021), p. 10. ISSN: 2227-9709. DOI: 10.3390/INFORMATICS8010010. URL: <https://www.mdpi.com/2227-9709/8/1/10/html><https://www.mdpi.com/2227-9709/8/1/10>.
- [37] Muhammad Ali et al. "MALGRA: Machine Learning and N-Gram Malware Feature Extraction and Detection System". In: *Electronics* 2020, Vol. 9, Page 1777 9 (11 Oct. 2020), p. 1777. ISSN: 2079-9292. DOI: 10.3390/ELECTRONICS9111777. URL: <https://www.mdpi.com/2079-9292/9/11/1777><https://www.mdpi.com/2079-9292/9/11/1777>.
- [38] Bander Ali Saleh Al-Rimy et al. "A Pseudo Feedback-Based Annotated TF-IDF Technique for Dynamic Crypto-Ransomware Pre-Encryption Boundary Delineation and Features Extraction". In: *IEEE Access* 8 (2020), pp. 140586–140598. ISSN: 21693536. DOI: 10.1109/ACCESS.2020.3012674.
- [39] Bander Ali Saleh Al-Rimy et al. "Redundancy Coefficient Gradual Up-weighting-based Mutual Information Feature Selection technique for Crypto-ransomware early detection". In: *Future Generation Computer Systems* 115 (Feb. 2021), pp. 641–658. ISSN: 0167-739X. DOI: 10.1016/J.FUTURE.2020.10.002.
- [40] Sunoh Choi et al. "Attention-Based Automated Feature Extraction for Malware Analysis". In: *Sensors* 2020, Vol. 20, Page 2893 20 (10 May 2020), p. 2893. ISSN: 1424-8220. DOI: 10.3390/S20102893. URL: <https://www.mdpi.com/1424-8220/20/10/2893><https://www.mdpi.com/1424-8220/20/10/2893>.
- [41] Tanya Gera et al. "Dominant Feature Selection and Machine Learning-Based Hybrid Approach to Analyze Android Ransomware". In: *Security and Communication Networks* 2021 (2021). ISSN: 19390122. DOI: 10.1155/2021/7035233.
- [42] Muhammad Furqan Rafique et al. "Malware Classification using Deep Learning based Feature Extraction and Wrapper based Feature Selection Technique". In: (Oct. 2019). URL: <https://arxiv.org/abs/1910.10958v3>.
- [43] Arslan Ashraf et al. "Ransomware Analysis using Feature Engineering and Deep Neural Networks". In: (Oct. 2019). URL: <https://arxiv.org/abs/1910.00286v2>.
- [44] Sana Aurangzeb et al. "BigRC-EML: big-data based ransomware classification using ensemble machine learning". In: *Cluster Computing* 25 (5 Oct. 2022), pp. 3405–3422. ISSN: 15737543. DOI: 10.1007/S10586-022-03569-4/TABLES/11. URL: <https://link.springer.com/article/10.1007/s10586-022-03569-4>.
- [45] Guoqing Xiao et al. "MalFCS: An effective malware classification framework with automated feature extraction based on deep convolutional neural networks". In: *Journal of Parallel and Distributed Computing* 141 (July 2020), pp. 49–58. ISSN: 0743-7315. DOI: 10.1016/J.JPDC.2020.03.012.
- [46] Danish Vasan et al. "Image-Based malware classification using ensemble of CNN architectures (IMCEC)". In: *Computers & Security* 92 (May 2020), p. 101748. ISSN: 0167-4048. DOI: 10.1016/J.COSE.2020.101748.

- [47] Jinpei Yan, Yong Qi, and Qifan Rao. "Detecting Malware with an Ensemble Method Based on Deep Neural Network". In: *Security and Communication Networks* 2018 (2018). issn: 19390122. doi: 10.1155/2018/7247095.
- [48] Mouhammd Al-Kasassbeh et al. "Feature selection using a machine learning to classify a malware". In: *Handbook of Computer Networks and Cyber Security: Principles and Paradigms* (Jan. 2019), pp. 889–904. doi: 10.1007/978-3-030-22277-2_36.
- [49] Giacomo Iadarola et al. "Image-based malware family detection: An assessment between feature extraction and classification techniques". In: *IoTBDs 2020 - Proceedings of the 5th International Conference on Internet of Things, Big Data and Security* (2020), pp. 499–506. doi: 10.5220/0009817804990506. url: https://www.researchgate.net/publication/341474538_Image-based_Malware_Family_Detection_An_Assessment_between_Feature_Extraction_and_Classification_Techniques <https://www.scitepress.org/Papers/2020/98178/98178.pdf>.
- [50] Tareq M. Shami et al. "Particle Swarm Optimization: A Comprehensive Survey". In: *IEEE Access* 10 (2022), pp. 10031–10061. issn: 21693536. doi: 10.1109/ACCESS.2022.3142859.
- [51] Muhammad Shabbir Abbasi, Harith Al-Sahaf, and Ian Welch. "Particle Swarm Optimization: A Wrapper-Based Feature Selection Method for Ransomware Detection and Classification". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 12104 LNCS (2020), pp. 181–196. issn: 16113349. doi: 10.1007/978-3-030-43722-0_12/COVER. url: https://link.springer.com/chapter/10.1007/978-3-030-43722-0_12.
- [52] Muhammad Shabbir Abbasi et al. "Behavior-based ransomware classification: A particle swarm optimization wrapper-based approach for feature selection". In: *Applied Soft Computing* 121 (2022), p. 108744. issn: 1568-4946. doi: <https://doi.org/10.1016/j.asoc.2022.108744>. url: <https://www.sciencedirect.com/science/article/pii/S1568494622001867>.
- [53] Ban Mohammed Khammas. "Ransomware Detection using Random Forest Technique". In: *ICT Express* 6 (4 Dec. 2020), pp. 325–331. issn: 2405-9595. doi: 10.1016/j.icte.2020.11.001.
- [54] Ebtesam J. Alqahtani, Rachid Zagrouba, and Abdullah Almuhaideb. "A survey on android malware detection techniques using machine learning Algorithms". In: *2019 6th International Conference on Software Defined Systems, SDS 2019* (June 2019), pp. 110–117. doi: 10.1109/SDS.2019.8768729.
- [55] Mohammad Masum et al. "Ransomware Classification and Detection with Machine Learning Algorithms". In: *2022 IEEE 12th Annual Computing and Communication Workshop and Conference, CCWC 2022* (2022), pp. 316–322. doi: 10.1109/CCWC54503.2022.9720869.
- [56] Fakhroddin Noorbehbahani, Farzaneh Rasouli, and Mohammad Saberi. "Analysis of machine learning techniques for ransomware detection". In: *Proceedings of 16th International ISC Conference on Information Security and Cryptology, ISCISC 2019* (Aug. 2019), pp. 128–133. doi: 10.1109/ISCISC48546.2019.8985139.
- [57] Arash Habibi Lashkari et al. "Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification". In: *Proceedings - International Carnahan Conference on Security Technology* 2018-October (Dec. 2018). issn: 10716572. doi: 10.1109/CCST.2018.8585560.
- [58] Usman Ahmed, Jerry Chun Wei Lin, and Gautam Srivastava. "Mitigating adversarial evasion attacks of ransomware using ensemble learning". In: *Computers and Electrical Engineering* 100 (May 2022), p. 107903. issn: 0045-7906. doi: 10.1016/j.compeleceng.2022.107903.
- [59] Maxat Akbanov, Vassilios G. Vassilakis, and Michael D. Logothetis. "Ransomware detection and mitigation using software-defined networking: The case of WannaCry". In: *Computers & Electrical Engineering* 76 (June 2019), pp. 111–121. issn: 0045-7906. doi: 10.1016/j.compeleceng.2019.03.012.
- [60] Ahmad O Almashhadani et al. "A Multi-Classifer Network-Based Crypto Ransomware Detection System: A Case Study of Locky Ransomware". In: *IEEE Access* 7 (2019), pp. 47053–47067. doi: 10.1109/ACCESS.2019.2907485.
- [61] Krzysztof Cabaj, Marcin Gregorczyk, and Wojciech Mazurczyk. "Software-defined networking-based crypto ransomware detection using HTTP traffic characteristics". In: *Computers & Electrical Engineering* 66 (Feb. 2018), pp. 353–368. issn: 0045-7906. doi: 10.1016/j.compeleceng.2017.10.012.

- [62] Greg Cusack, Oliver Michel, and Eric Keller. "Machine Learning-Based Detection of Ransomware Using SDN". In: *Proceedings of the 2018 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization*. Association for Computing Machinery, 2018, pp. 1–6. ISBN: 9781450356350. DOI: 10.1145/3180465.3180467. URL: <https://doi.org/10.1145/3180465.3180467>.
- [63] Omar M K Alhawi, James Baldwin, and Ali Dehghantanha. "Leveraging Machine Learning Techniques for Windows Ransomware Network Traffic Detection". In: ed. by Ali Dehghantanha, Mauro Conti, and Tooska Dargahi. Springer International Publishing, 2018, pp. 93–106. ISBN: 978-3-319-73951-9. DOI: 10.1007/978-3-319-73951-9_5. URL: https://doi.org/10.1007/978-3-319-73951-9_5.
- [64] Daniel Morato et al. "Ransomware early detection by the analysis of file sharing traffic". In: *Journal of Network and Computer Applications* 124 (Dec. 2018), pp. 14–32. ISSN: 1084-8045. DOI: 10.1016/J.JNCA.2018.09.013.
- [65] Yu Lun Wan et al. "Feature-Selection-Based Ransomware Detection with Machine Learning of Data Analysis". In: *2018 3rd International Conference on Computer and Communication Systems, ICCCS 2018* (Sept. 2018), pp. 392–396. DOI: 10.1109/CCOMS.2018.8463300.
- [66] Pagán Jr and Khaled Elleithy. *A Multi-Layered Defense Approach to Safeguard Against Ransomware*. Jan. 2021, pp. 942–947. DOI: 10.1109/CCWC51732.2021.9375988.
- [67] Manveer Patyal et al. "Multi-layered defense architecture against ransomware". In: *International Journal of Business and Cyber Security* 1 (Jan. 2017). URL: https://www.researchgate.net/publication/315471509_Multi-layered_defense_architecture_against_ransomware.
- [68] Anh Pham Tuan et al. "Malware Detection PE-Based Analysis Using Deep Learning Algorithm Dataset". In: (May 2018). DOI: 10.6084/m9.figshare.6635642.v1. URL: https://figshare.com/articles/dataset/Malware_Detection_PE_Based_Analysis_Using_Deep_Learning_Algorithm_Dataset/6635642.
- [69] Daniel Gibert. "PE Parser: A Python package for Portable Executable files processing". In: *Software Impacts* 13 (2022), p. 100365. ISSN: 2665-9638. DOI: <https://doi.org/10.1016/j.simpa.2022.100365>. URL: <https://www.sciencedirect.com/science/article/pii/S2665963822000756>.
- [70] Neiler W. Pérez-Díaz et al. "Ransomware Identification Through Sandbox Environment". In: *Lecture Notes in Networks and Systems* 560 LNNS (2023), pp. 326–335. ISSN: 23673389. DOI: 10.1007/978-3-031-18458-1_23/COVER. URL: https://link.springer.com/chapter/10.1007/978-3-031-18458-1_23.
- [71] Tamas K. Lengyel et al. "Scalability, Fidelity and Stealth in the DRAKVUF Dynamic Malware Analysis System". In: *Proceedings of the 30th Annual Computer Security Applications Conference*. 2014. DOI: <https://doi.org/10.1145/2664243.2664252>. URL: https://www.acsac.org/2014/program-final/oc_multifile/3/52.pdf.
- [72] A. Alfred Raja Melvin and G. Jaspher W. Kathrine. "A quest for best: A detailed comparison between drakvuf-vmi-based and cuckoo sandbox-based technique for dynamic malware analysis". In: *Advances in Intelligent Systems and Computing* 1167 (2021), pp. 275–290. ISSN: 21945365. DOI: 10.1007/978-981-15-5285-4_27/COVER. URL: https://link.springer.com/chapter/10.1007/978-981-15-5285-4_27.
- [73] Pfizer documents — Kaggle. URL: <https://www.kaggle.com/datasets/konradb/pfizer-documents?resource=download>.
- [74] Lester James V. Miranda. "PySwarms: a research toolkit for Particle Swarm Optimization in Python". In: *Journal of Open Source Software* 3 (21 Jan. 2018), p. 433. ISSN: 2475-9066. DOI: 10.21105/JOSS.00433. URL: <https://joss.theoj.org/papers/10.21105/joss.00433>.
- [75] F Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830. DOI: <https://doi.org/10.48550/arXiv.1201.0490>.